INDEX

NAME: Mohamed Humaid STD.: SEC.: ROLL NO.: 240701322 SUB.:

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	20/7/26	Study of various network commands used in linux and windows		
2.	21/7/26	Study of different types of network cables		
3.	27/7/26	Study the packet trace tool installation and user interface overview.		
4.	28/7/26	Setting up a LAN		
5.	4/8/26	Packet Capture Tool : Wireshark		
6.	11/8/26	Error detection and correction using hamming code.		
7.	24/8/26	Flow control at data link layer using sliding window protocol.		
8.	31/8/26	Discover live hosts using nmap on TryHackMe.		
9.	8/9/26	Subnetting in Cisco Packet Tracer		
10.	15/9/26	Configure a Single internetwork using a router by Cisco Packet Tracer		
11	28/9/26	Simulate RIP using Cisco packet Tracer.		
12a	29/9/26	Echo client-server application using TCP/UDP		

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
12b	6/10/25	Chat client-server application using TCP/UDP sockets		
13	12/10/25	Ping program		
14	12/10/25	Program using raw sockets.		

Completed
13/10/25

Study of various network commands used in Linux and Windows

Aim:
To study the various network commands used in linux and windows.

Networking commands

1. arp -a

Description:
ARP is short form of address resolution protocol, it will shows the IP address of your computer along with the IP address and MAC address of your router.

Output:
- gateway (10.0.2.2) at 52:55:0a:00:02:02 [ether] on enp0s3

2. hostname

Description:
This is the simplest of all TCP/IP commands. It simply displays the name of your computer.

Output:
fedora

3. ipconfig /all

Description:
This command displays detailed configuration information about your TCP/IP connection including routes, gateway, DNS, DHCP and type of ethernet adapter in your system.

Output:

bash: ipconfig /all: No such file or directory

4. netstat

Description:
netstat displays a variety of statistics about a computers active TCP/IP connections. It is a command line tool for monitoring network connections both incoming and outgoing as well as viewing routing tables, interface statistics etc.

Output:
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address Foreign Address State
udp 0 0 fedora:bootpc _gateway:bootps ESTABLISHED
Active UNIX domain sockets (w/o servers)
Proto RefCnt Flags Type State I-Node Path
unix 3 [] STREAM CONNECTED 29827

5. nbtstat -a

Description:
This command helps solve problems with NetBIOS name resolution (Nbt stands for NetBIOS over TCP/IP).

Output:
Displays protocol statistics and current TCP/IP connections using NBT
NBTSTAT [[-a RemoteName] [-A IP address] [-c] [-n] [-r] [-R] [-RR] [-s] [interval]]

6. nslookup

Description:
name server lookup is a tool used to perform DNS lookups in Linux. It is used to display DNS details, such as the IP address of a particular computer, the MX records for a domain or the NS servers of a domain. nslookup can operate in two modes: interactive and non-interactive.

Output:
Server: 127.0.0.53
Address: 127.0.0.53#53
Non-authoritative answer:
Name: www.google.com
Address: 142.251.156.119

7. pathping

Description:
Pathping is unique to window's, and is basically a combination of the ping and tracert commands. Pathping traces the route to the destination address then launches a 25 second test of each router along the way.

Output:
Usage: pathping [-g host-list] [-h maximum_hops] [-i address] [-n] [-p period] [-q num_queries] [-w timeout] [-4] [-6] target_name

8. ping

Description:
Packet INternet Groper command is the best way to test connectivity between two nodes. Ping use ICMP to communicate to other devices.

Output:
~~64 bytes from localhost (::1): icmp_seq~~ Fedora
64 bytes from fedora (fe80:90ff:454d:ce50): icmp_seq=1 ttl=64 time=0.058 ms

PING 24.21.4.8 (24.21.4.8) 56(84) bytes of data

64 bytes from 142.251.152.119: icmp_seq=1 ttl=255 time=10.7 ms

9. Route

Description:
route command is used to show/manipulate the IP routing table. It is primarily used to setup static routes to specific host or networks via an interface.

Output:
Kernal IP routing table

Destination	Gateway	Genmask	Flags	Metric	Ref	Use
default	_gateway	0.0.0.0	UG	100	0	0
10.0.2.0	0.0.0.0	255.255.255.0	U	100	0	0

Iface
enp0s3
enp0s3

nbtstat -c
Description:
Displays the NetBIOS name cache, which contains recently resolved NETBIOS.

Output:
NetBIOS Remote Cache Name Table

Name	Type	Host Address	Life (sec)
SERVER01 <20>	UNIQUE	192.168.1.10	450

nbtstat -r
Description:
Displays NetBIOS name resolution statistics.
Output:
NetBIOS Names Resolution and Registration Statistics
Resolved By Broadcast = 12
Resolved By Name Server (WINS) = 18.

Some important linux networking commands.

1. ip

The ip command is one of the basic commands every administrator will need in daily work, from setting up new systems and assigning IPs to troubleshooting existing systems. The ip command can show address information, manipulate routing, plus display network various devices, interfaces and tunnels.

Here are some common use cases for the ip command.

a. ip address show

Description:
To show the IP address assigned to an interface on your server.

Output:
1: lo: <LOOPBACK, UP, LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo

b. To assign an IP to any interface, for example enps03:
a. [root@sever~] # ip address add 192.168.1.254/24 dev enps03

Output:
No output if the command is successful

c. To delete an IP on an interface
a. [root@sever~] # ip address del 192.168.1.254/24 dev enps03

Output:
No output if successful.

d. Alter the status of the interface by bringing the interface eth0 online: [root@sever~] # ip link set eth0 up

Output:
No output if successful

e. Alter the status of the interface by bringing the interface eth0 offline: [root@sever~] # ip link set eth0 promisc on down

Output:
No output if successful

f. Alter the status of the interface by enabling promiscuous mode for eth0: [root@sever~] # ip link set eth0 promisc on

Output:
No output if successful

g) Add a default route (for all addresses) via the local gateway 192.168.1.254 that can be reached on device eth0:
[root@server~] #ip route add default via 192.168.1.254 dev eth0

Output:
No output if successful

h) Add a route to 192.168.1.0/24 via the gateway at 192.168.1.254
[root@server~] #ip route add 192.168.1.0/24 via 192.168.1.254

Output:
No output if successful.

i) Add a route to 192.168.1.0/24 that can be reached on device eth0: [root@server~] #ip route add 192.168.1.0/24 dev eth0

Output:
No output if successful

j) Delete the route for 192.168.1.0/24 via the gateway at 192.168.1.254. [root@server~] # ip route delete 192.168.1.0/24 via 192.168.1.254

Output:
No output if successful.

k) Display the route taken for IP 10.10.1.4:
[root@server~] #ip route get 10.10.1.4.

Output:
10.10.1.4 via 192.168.1.254 dev eth0 src 192.168.1.100

cache

2. ifconfig

The ifconfig command was is a staple in many sysadmin's tool belt for configuring and troubleshooting networks. It has since been replaced by the ip command discussed above.

Output:

```
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>mtu 1500
      inet 192.168.1.100  netmask 255.255.255.0
      ether 08:00:27:4f:26:11

lo: flags=73<UP,LOOPBACK,RUNNING>mtu 65536
    inet 127.0.0.1  netmask 255.0.0.0
```

3. mtr

MTR is a program with a command line interface that serves as a network diagnostic and troubleshooting tool. This command combines the functionality of the ping and traceroute commands. Just like a traceroute, the mtr command will show the route from a computer to a specified host. mtr provides a lot of statistics about each hop, such as response time and percentage. With the mtr command, you will get more information about the route and be able to see problematic devices along the way. If you see a sudden increase in response time or packet loss, then obviously, there is a bad link somewhere.

The syntax of the command is as follows:

mtr <options> hostname/IP

a. The basic mtr command shows you the statistics, including each hop (hostnames) with time and loss%.

[root@sever ~] # mtr google.com

Output:

fedora (10.0.2.15) → google.com (142.250.207.78)

keys: Help Display mode Restart statistics Order of fields quit

	Host	Packets Loss%	Snt	Last	Avg	Pings Best	Wrst	StDev
1.	pnmaa-bd-in-f14.1e100.net	1.7%	125	98.7	90.8	8.8	796.6	110.6

b. Show numeric IP addresses (if you use -g you will get IP addresses (numbers) instead of hostnames):

[root@sever ~] # mtr -g google.com

Output:

Usage:

mtr [options] hostname

-F, --filename FILE read hostname(s) from a file

-4 use IPV4 only

-u --udp use IPV6 only

-T --tcp use TCP instead of ICMP echo.

c. show the numeric IP addresses and hostnames,
~~too~~ : [root@sever ~] # mtr -b google.com

Output:
fedora (10.0.2.15) -> google.com (142.251.221.206)
keys: Help Display mode Restart statistics Order of statistics
Host Packets Pings
loss% Snt Last Avg Best Wrst StDev
1. pnmaa-ba- 1.2% 247 43.3 75.9 8.6 1121. 172.9
in-f14.1e100.
net (142.251.221.206)

d. Set the number of pings that you want to send : [root@sever ~] # mtr -c 10 google.com

Output:
fedora (10.0.2.15) -> google.com (142.251.221.206)
keys: Help Display mode Restart statistics Order of Statistic
Host Packets Pings
loss% Snt Last Avg Best Wrst St
1. pnmaa-ba- 1.5% 115 50 76 6 ~~35~~ 180.
in-f14-1e100.
net (142.251.221.206)

4. tcpdump

The tcpdump command is designed for capturing and displaying packets.

You can install tcdump with the command below:

[root @ server ~] # dnf install -y tcpdump

Before starting any capture, you need to know which interfaces tcpdump can use. You will need to use sudo or have root access in this case.

[root @ server ~] # tcpdump -D

If you want to capture traffic on eth0, you can initiate that with tcpdump -i eth0 sample output.

Output:

tcpdump: verbose output suppressed use -v or -vv for full protocol decode listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes.

Capture traffic to and from one host

You can filter out traffic coming from a specific host. For example, to find traffic coming from and going to 8.8.8.8, use the command:

```
[root@server ~] # tcpdump -i eth0 -c 10 host 8.8.8.8.
```

For traffic coming from 8.8.8.8, use:

```
[root@server ~] # tcpdump -i eth0 src host 8.8.8.8
```

For outbound traffic going to 8.8.8.8, use:

```
[root@server ~] # tcpdump -i eth0 dst host 8.8.8.8
```

Capture traffic to and from a network:

You can also capture traffic to and from a specific network using the command below:

```
[root@server ~] # tcpdump -i eth0 net 10.1.0.0
                  mask 255.255.255.0.
```

or

```
[root@server ~] # tcpdump -i eth0 net
                  10.1.0.0/24
```

Capture traffic to and from port numbers

Capture only DNS port 53 traffic:

```
[root@server ~] # tcpdump -i eth0 port 53.
```

For a specific host,

```
[root@server ~] # tcpdump -i eth0 host 8.8.8.8 and port 53.
```

To capture only HTTPS traffic,

[root@server~] # tcpdump -i eth0 -c 10
host www.google.com and port 443.

To capture all port except port 80 and 25,
[root@server~] # tcpdump -i eth0 port
not 53 and not 25

5. ping

Ping is a tool that verifies IP-level connectivity to another TCP/IP computer by sending ICMP echo request messages. The receipt of corresponding echo reply messages is displayed, along with round trip times. Ping is the primary TCP/IP command used to troubleshoot connectivity, reachability, and name resolution.

[root@server~] # ping google.com.

Output:
PING google.com (142.251.221.206) 56(84) bytes of data.
64 bytes from pnmaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=1 ttl=255 time=15.1 ms
64 bytes from pnmaa-ba-in-f14.1e100.net

(142.251.221.206): icmp-seq=2 ttl=255
time=22.5 ms

If you want to ping a host 10 times, use the following command:

[root@server~] #ping -c 10 google.com.

While pinging a host, you'll find different output from the ping results, including the following three examples.

Destination Host Unreachable
The possible best reason is there is no route from the local host system and to the destination desired destination host, or a remote router reports that it has no route to the destination host.

Request timed out
This result means that no echo reply messages were received within the default time of one second or the time that you set while you are pinging that host. This can be due to many different causes; the most common include network congestion, failure of the REP request, packet filtering/firewall etc.

Unknown host/Ping Request could NOT Find Host
Maybe you misspelled the hostname or the host does not exist at all in the network. You must have 0% packet loss for every ping result with a good latency or lower response time.

Depending on which transmission medium (UTP, fibre optic cable, Wi-Fi) you're using, your latency will differ.

Configuring an Ethernet connection by using nmcli

If you connect a host to the network over Ethernet, you can manage the connection's settings on the command line by using nmcli utility.

Procedure

1. List the NetworkManager connection profiles:

nmcli connection show

NAME	UUID	TYPE	DEVICE
Wired connection 1	a5eb6490-cc20-3668-81f8-0314a27f3f75	ethernet	enp1s0

2. # nmcli connection add con-name <connection-name> ifname <device-name> type ethernet

Skip this step to modify an existing profile.

3. Optional: Rename the connection profile:
nmcli connection modify "Wired connection 1"
Here, "Wired connection 1" is the name of the connection

4. Display the current settings of the connection profile: # nmcli connection show

```
connection.interface-name:
                        enp1s0
connection.autoconnect  yes
ipv4.method:            auto
ipv6.method             auto
```

5. Configure the IPv4 settings:
   - To use DHCP, enter:

#nmcli connection modify "Wired connection 1" ipv4.method auto.

Skip this step if ipv4.method is already set to auto (default).

   - To set a static IPv4 address, network mask, default gateway, DNS servers, and search domain, enter:

#nmcli connection modify "Wired connection 1" ipv4.method manual ipv4.addresses 192.0.2.1/24 ipv4.gateway 192.0.2.254 ipv4.dns 192.0.2.200 ipv4.dns-search example.com

6. Configure the IPv6 settings:
   - To use stateless address autoconfiguration (SLAAC), enter:

#nmcli connection modify "Wired connection 1" ipv6.method auto

Skip this step if ipv6.method is already set to auto (default)

- To set a static IPV6 address, network mask, default gateway, DNS servers, and search domain, enter:

nmcli connection modify "Wired connection 1" ipV6.method manual ipv6.addresses 2001:db8:1::fffe/64 ipv6.gateway 2001:db8:1::fffe ipV6.dns 2001:db8:1::ffbb ipV6.dns-search example.com.

7. Activate the profile:

nmcli connection up Internal-LAN

Verification

1. Display the IP settings of the NIC:

ip address show enp1s0

enp1s0: <BROADCAST, MULTICAST, UP, LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
link/ether 52:54:00:17:b8:b6 brd ff:ff:ff:ff:ff:ff
inet 192.0.2.1/24 brd 192.0.2.255 scope global noprefixroute enp1s0.

2. Display the IPV4 default

gateway # ip route show
default
default via 192.0.2.254 dev enp1s0 proto static metric 102.

3. Display the ~~result~~ IPV6 default

gateway : # ip-6route show
default

default via 2001:dbs:1::ffee dev enp1SD proto static
metric 102 pref medium

4. Display the DNS

Settings : # cat
Letcl resolv.conf
search exemple.com
nameserver 192.0.2.200
nameserver 2001:dbs:1::ffbb.
If multiple connection profiles are active at the same time, the order of nameserver entries depend on the DNS priority values in these profile and the connection types

5. Use the ping utility to verify that this host can send packets to other hosts # ping <host-name or IP address>

Troubleshooting

- Verify that the network cable is plugged in to the host and a switch.
- Check whether the link failure exists only on this host or also on other hosts connected to the same switch.
- Verify that the network cable and the network interface

are working as expected. Perform hardware diagnostic steps and replace defect cables and network interface cards.

- If the configuration on the disk does not match the configuration on the device, starting or restarting NetworkManager creates an in-memory connection that reflects the configuration of the device.

Questions

1. Which command is used to find the reachability of a host machine from your device?

Ans:- ping

2. Which command gives the details of the hops taken by a packet to reach its destination?

Ans:- traceroute (LINUX) or tracert (Windows)

3. Which command displays the IP configuration of your machine?

Ans:-

- LINUX: ip addr (or ifconfig on older systems)
- Windows: ipconfig

4. Which command displays the TCP port status on your machine?

Ans:- netstat -t

5. Write the command to modify the IP configuration in a LINUX machine.

Ans:- sudo ip addr add 192.168.1.100/24 dev eth0 .

CamScanner

classmate
Date
Page

Practical 2

Aim:
To study different types of network cables.

a) Understand different types of network cable.

Different type of cables used in networking are:

1. Unshielded Twisted Pair (UTP) cable
2. Shielded Twisted Pair (STP) cable
3. Coaxial cable
4. Fibre optic cable

Cable Type	Category	Maximum Data Transmission	Advantages/ Disadvantages	Application/ use	Image
UTP	Category 3	10 bps	Advantages • cheaper in cost • Easy to install as they have a	10 Base-T Ethernet	
	Category 5	upto 100 Mbps	smaller overall diameter. Disadvantages • More prone	Fast Ethernet, Gigabit Ethernet	
	Category 5e	1 Gbps	to (EMI) electro-magnetic inter-ference and noise.	Fast Ethernet/ Gigabit Ethernet	

CamScanner

STP	Category 6, 6a	10 Gbps	Advantages • Shielded • Faster than UTP. • less susceptible to noise and interference. Disadvantages • Expensive	1gabit Ethernet, 10G Ethernet (55m). widely used in data centres.
SSTP	~~SSTP~~ Category 7	10Gbps	• Greater installation effort	Gigabit Ethernet, 10G Ethernet (100m)
Co-axial cable	RG-6 RG-59 RG-11	10-100Mbps	Advantages • High bandwidth • Immune to interference • low bandwidth • Versatile Disadvantages • limited distance • cost • Size is bulky.	Speed of signal is 500m Television network High speed internet connections







Fibre optics cable	Single mode Multimode	100 Gbps	Advantages • High speed • High bandwidth • High security • long distance Disadvantages • Expensive • requires skilled installers	• Maximum distance of fibre optics cable is around 100 meters.



b) Make your own ethernet cross-over cable / straight cable.

Tools and parts needed:

- Ethernet cabling. CAT5e is certified for gigabit support, but CAT5 cabling works as well, just over shorter distances.
- A crimping tool. This is an all-in-one networking tool shaped to push down the pins in the plug and strip and cut the shielding of the cables.
- optional two plug shields.

Step1:
To start construction of the device, begin by threading shields onto the cable.

Step2:
Next, strip approximately 15cm of cable shielding from both ends. The crimping tool has a round area to complete this task.

Step3:
After, you will need to untangle the wires; there should be four "twisted pairs." Referencing back to the sheet, change them from top to bottom. One end should be in arrangement A and the other in B.

Step4:
Once the order is correct, bunch them together in a line, and if there are any that stick out farther than others, snip them back to create an even level. The difficult aspect is placing these into the RJ45 plug without messing up the order. To do so, hold the plug with the clip side facing away from you and have the gold pins facing toward you, as shown.

Step5:
Next, push the cable right in. The notch at the end of the plug needs to be just over to the cable shielding, and if it isn't, that means you stripped off too much shielding. Simply snip the cables back a little more.

Step5:
After the wires are securely sitting inside the plug, insert into the crimping tool and push down. It should be shaped correctly, but pushing too hard can crack the fragile plastic plug.

Step7:
Lastly, repeat for the other end using diagram B (to make crossover cables) / using diagram A (to make a straight through cable). To test it, plug it in and attempt to connect two devices directly.

Questions

1. What is the difference between cross cable and straight cable?

Ans:- Straight cable: Same wiring standard is used at both ends (T568A - T568A or T568B - T568B).

Cross cable: Different wiring standards are used at each end (T568A - T568B), so the transmit and receive pairs are crossed.

2. Which type of cable is used to connect two PC?

Ans:- Cross cable

3. Which type cable is used to connect a router/ switch to your PC?

Ans: Straight cable

4. Find out the category of twisted pair cable used in your lab to connect the PC to the network socket.

Ans: Category 5e

5. Write down your understanding, challenges faced and output received while making a twisted pair/ straight cable.

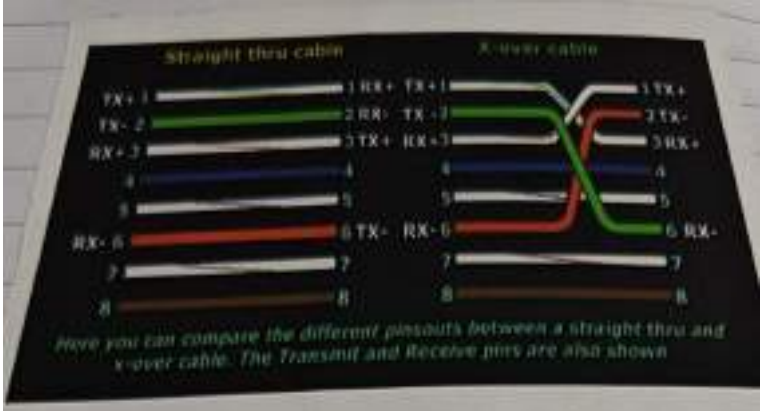
I learned how to make twisted-pair ethernet cables and understood the difference between straight and cross cables. I also learned about T568A and T568B wiring standards.

Challenges faced:

- Arranging the wires in the correct color sequence.
- Keeping the wires straight and in the correct order.

Output:

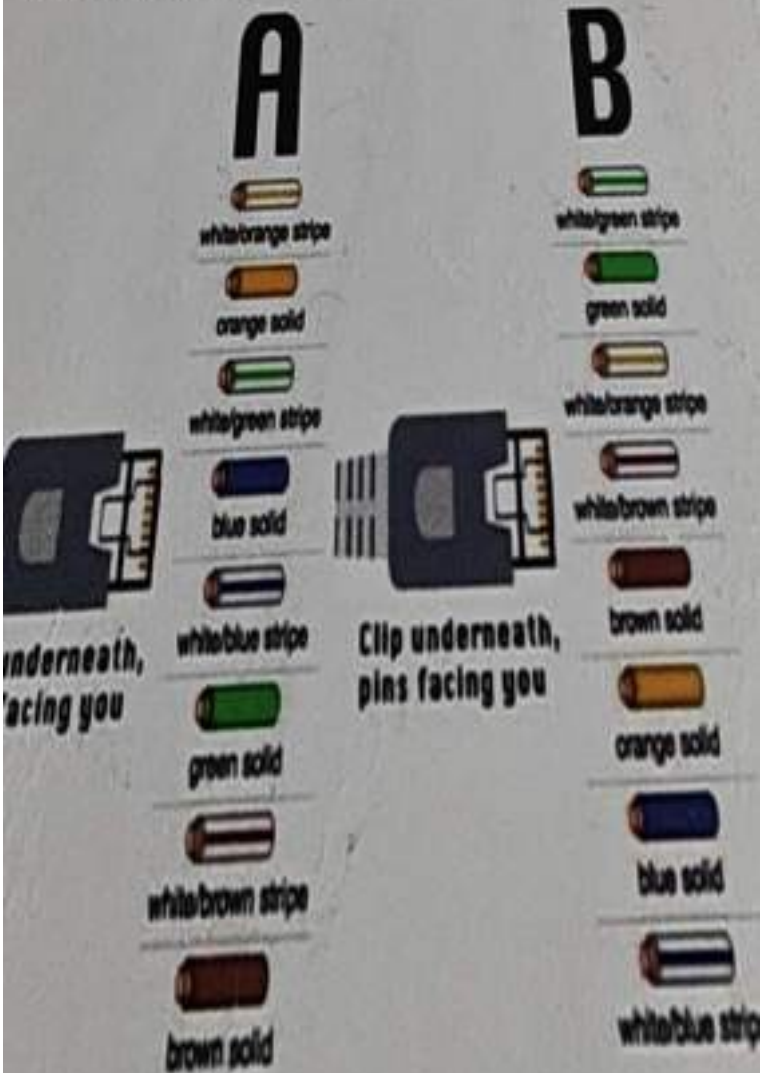
The cable was successfully crimped and tested using a cable tester. The tester showed the correct sequence/ continuity of all 8 wires, indicating that the cable was working properly.



Difference blw crossover cable and St

cable

Straight through network cable: both sides should be A
Crossover cable: One side A, one side B



CamScanner

Practical - 3

Aim:
To study the packet tracer tool installation and user interface overview.

e) To understand environment of CISCO PACKET TRACER to design simple network.

Introduction
A simulator, as the name suggests, simulates network devices and its environment. Packet tracer is an exciting network design, simulation and modelling tool.

1. It allows you to model complex systems without the need for dedicated equipment.
2. It helps you to practice your network configuration and troubleshooting skills via computer or an android or iOS based mobile device.
3. It is available for both the Linux and Windows desktop environments.
4. Protocols in Packet Tracer are coded to work and behave in the same way as they would on real hardware.

User Interface Overview:
The layout of packet tracer is divided into several components. The components of the packet tracer interface are as follows: match the numbering with explanations.

1. Menu Bar - This is a common menu found in all software applications; it is used to open, save, print, change preferences, and so on.

2. Main toolbar - This bar provides shortcut icons to menu options that are commonly accessed, such as open, save, zoom, undo, and redo, and on the right-hand side is an icon for entering network information for the current network.

3. Logical / Physical workspace tabs - These tabs allow you to toggle between the logical and physical work areas.

4. Workspace - This is the area where topologies are created and simulations are displayed.

5. Common tools bar : This toolbar provides controls for manipulating topologies, such as select, move layout, place note, delete, inspect, resize shape, and add simple/complex PDU.

6. Real-time / Simulation tabs - These tabs are used to toggle between the real and simulation modes. Buttons are also provided to control the time, and to capture the packets.

7. Network component box - This component contains all of the network and end devices available with packet tracer, and is further divided into 2 stages: Area 7a: Device - type selection box - This area contains

device categories Area 7b: Device-specific selection box - When a device category is selected, this selection box displays the different device models within that category.

8. User-created packet box - Users can create highly customized packets to test their topology from this area, and the results are displayed as a list.

d) Analyze the behaviour of network devices using CISCO PACKET TRACER Simulator.

1. From the network component box, click and drag the below components:
   a. 4 Generic PCs and One HUB.
   b. 4 Generic PCs and One switch.

2. Click on Connections:
a. Click on copper straight-through cable.
b. Select one of the PC and connect it to HUB using the cable. The link LED should glow in green, indicating that the link is up. Similarly connect remaining 3 PCs to the HUB.
c. Similarly connect 4 PCs to the switch using copper-through cable.

3. Click on the PC's connected to hub, go to the Desktop tab, click on IP configuration, and enter an IP address and subnet mask. Here, the

default gateway and DNS server information is not needed as these are only two end devices in the network.

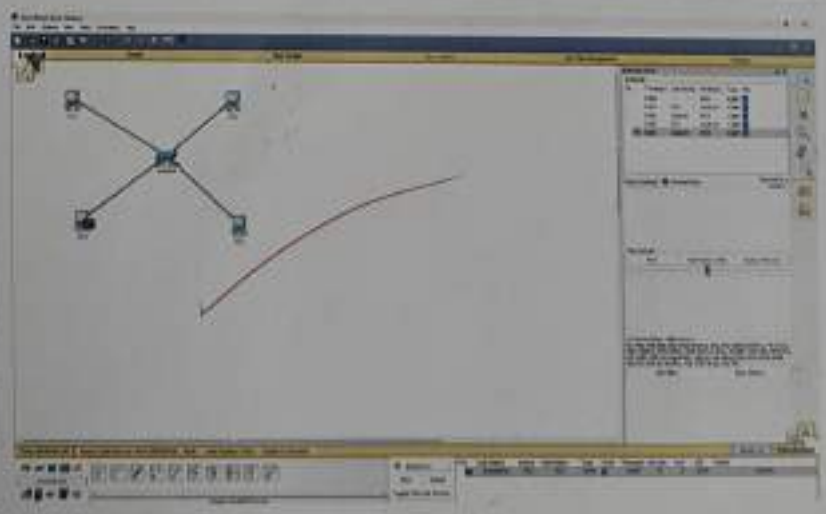
Click on the PDU (message icon) from the common tool bar,

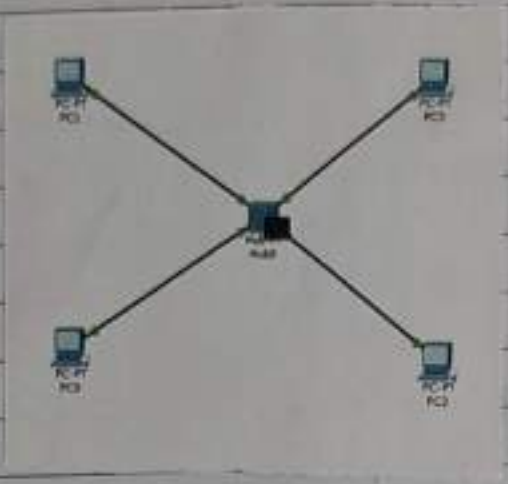
a. Drag and drop it on one of PC (source machine) and then drop it on another PC (destination machine) connected to the HUB.

4. Observe the flow of PDU from source PC to destination PC by selecting the real time mode of simulation.

5. Repeat step # 3 to step # 5 for the PC's connected to the switch.

6. Observe how hub and switch are forwarding the PDU and write your observation and conclusion about the behaviours of switch and HUB.





Questions

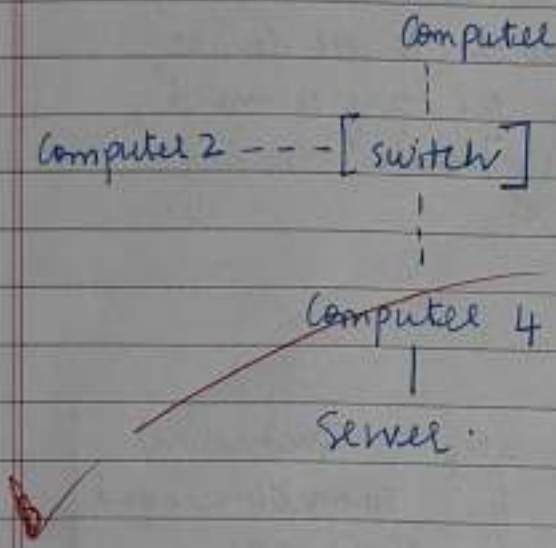
a) From your observation write down the behaviour of switch and HUB in terms of forwarding the packets received by them.

- Switch : A switch forwards the received packet only to the specific port / device for which the packet is intended, based on its MAC address table.

- Hub: A hub forwards the received packet to all ports / devices connected to it, regardless of the destination.

b. Find out the network topology implemented in your college and draw and label that topology in your observation book.

Topology - Star topology



Practical 4
Setting Up a Local Area Network (LAN)

1. Aim:
To design, set up and configure a LAN connecting four computers through a network switch, assign static IP addresses to all devices, verify connectivity using the ping command, and demonstrate resource/data sharing across the network.

2. Equipment Required

S.No	Experiment	Qty	Specifications
1	Personal Computers (PC)	4	Ethernet (NIC) enabled
2	Network Switch	1	8/16/24 port, unmanaged/managed
3	Ethernet Cables (Twisted, Cat 6)	4	Straight-through, RJ-45 connectors

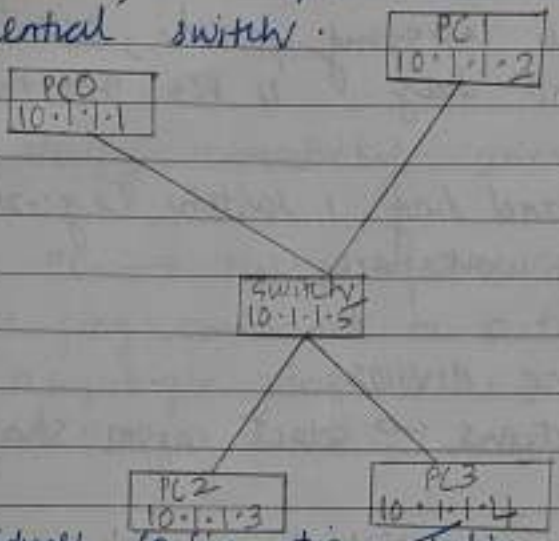
3. Theory
A LAN is a network that connects computers and devices within a limited geographical area such as a home, office, or laboratory allowing them to share resources such as files, printers and internet connections.

A star topology is used in this experiment, where all end devices (PCs) are individually connected to a central network switch. The switch operates at layer 2 of the OSI model and forwards data frames between connected

devices based on their MAC addresses. Every device on a LAN must be assigned an IP address within the same network range (as determined by the subnet mask) to be able to communicate with one another.

4. Network Diagram.

The LAN was implemented using a Star Topology as shown below, with 4 PCs connected individually to a central switch.



Star Topology LAN
(4 PCs + Switch)

5. IP Address Configuration Table

Device	IP address	Subnet Mask	Connected To
PC0	10.1.1.1	255.0.0.0	Switch Port Fa0/1
PC1	10.1.1.2	255.0.0.0	Switch Port Fa0/2
PC2	10.1.1.3	255.0.0.0	Switch Port Fa0/3
PC3	10.1.1.4	255.0.0.0	Switch Port Fa0/4
Switch (Management)	10.1.1.5	255.0.0.0	VLAN 1 Interface

6. Implementation in Cisco Packet Tracer.

Since physical hardware was limited, the LAN was also implemented and verified using Cisco Packet Tracer simulation software, following the steps below.

Step 1: Place the devices.

1. Open Cisco Packet Tracer
2. From the device category panel, select End Devices and drag 4 PCs onto the workspace.
3. Select switches and drag 1 switch (e.g.:-2960 series) onto the workspace.

Step 2: Cable the devices

4. Click the connections → select copper straight-through cable.
5. Click each PC, then click the switch, and select an available FastEthernet port to complete each connection.
6. Repeat for all 4 PC's (ports Fa0/1 to Fa0/4). Green link lights confirm a successful physical connection.

Step 3: Assign IP addresses to PCs.

7. Click a PC → Desktop tab → IP Configuration.
8. Enter the static IP address and subnet mask as per the IP table (Section 5) for each PC.

Step 4: Configure the switch Management IP (optional)

9. Click the switch → CLI tab.
10. Enter the following commands to assign a management IP:

```
enable
configure terminal
interface vlan 1
ip address 10.1.1.5 255.0.0.0
no shutdown
exit
```

Step 5: Verify connectivity using ping.

11. Click a PC → Desktop tab → Command prompt.
12. Use the ping command to test connectivity to the other PCs, e.g:- ping 10.1.1.2.
13. Successful "Reply from..." messages confirm connectivity.

Step 6: Demonstrate data sharing (Server Simulation)

14. Add a server device (End devices → Server-PT) and connect it to the switch.
15. Assign it an IP address (eg:- 10.1.1.10, subnet mask 255.0.0.0) via its desktop → IP configuration tab.
16. Open the server's services (Config) tab and enable the HTTP service.
17. From another PC, open the web browser (Desktop tab) and enter the server's IP address to load the page, confirming successful data transfer across the LAN.

Step 7: Observe packet flow in simulation mode.

18. Switch to simulation mode (bottom-right of the workspace).

19. Send a ping or web request between two devices and use the play/capture controls to step through the packet as it travels hop-by-hop through the switch, observing MAC address based forwarding.

7. Observation

All four PCs were successfully connected to the switch in a star topology and assigned unique static IP addresses within the same subnet (10.0.0.0/255.0.0.0). Ping tests between all devices returned successful replies, confirming end-to-end connectivity. The shared folder/server resource was successfully accessed from other computers on the network, confirming that the LAN was functioning correctly for resource sharing.

Practical 5

Packet Capture Tool: Wireshark

Aim:

* To perform experiments on packet capture tool: wireshark.

Packet Sniffer

- Sniffs messages being sent / received from / by your computer.
- Store and display the contents of the various protocol fields in the messages.
- Passive program
  - never sends packets itself.
  - no packets addressed to it.
  - receives a copy of all packets (sent / received)

Packet Sniffer Structure Diagnostic Tools

- Tcpdump
  - E.g:- tcpdump -enx host 10.129.41.2 -w exe3.out
- Wireshark
  - wireshark -r exe3.out

Description:

Wireshark

Wireshark, a network analysis tool formerly known as Ethereal, captures packets in real time and display them in human-readable format. Wireshark includes filters, color coding and other features that let you dig deep into network traffic and inspect individual packets. You can use wireshark to inspect a suspicious program's network traffic, analyze the traffic flow on your network, or troubleshoot network problems.

What we can do with wireshark:

- Capture network traffic
- Decode filters - capture and display
- Decode packet protocols using dissectors.
- Watch smart statistics
- Analyze problems
- Interactively browse that traffic.

Wireshark used for:

- Network administrators: troubleshoot network problems.
- Network security engineers: examine security problems.
- Developers: debug protocol implementations.
- People: learn network protocol internals.

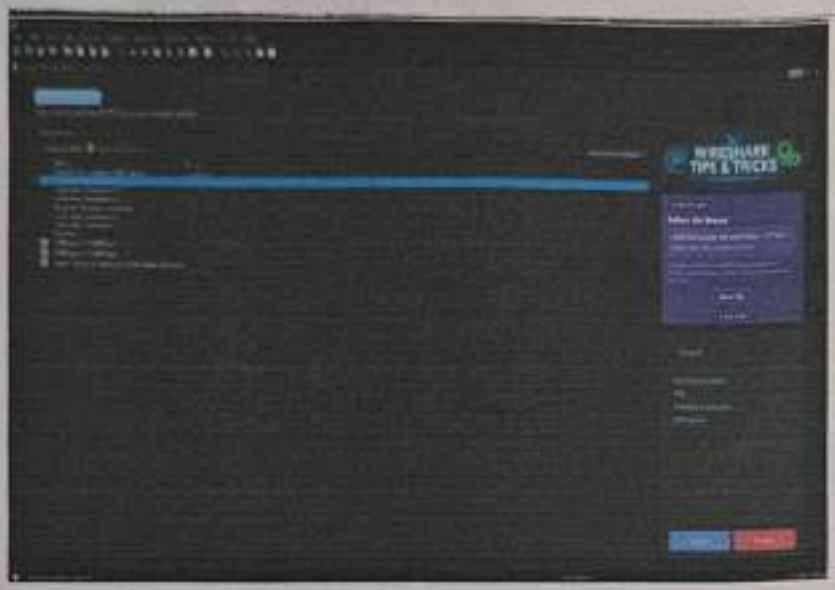
Capturing Packets

After downloading and installing wireshark, launch it and double-click the name of a network interface under capture to start capturing packets on that interface.
As soon as you click the interface's name, you'll see the packets start to appear in real time. Wireshark captures each packet sent to or from your system.

Procedure

- Select the local area connection in wireshark.
- Go to capture option.
- Select capture and stop automatically after 100 packets.
- Then click start capture.
- Save packets.
- Create a filter to display only TCP/UDP packets, inspect and provide the flow graph.
- Go to search bar at the top of the tool.
- Search TCP packets in search bar.
- To see the graph click statistics flowgraph.
- Create a filter for ARP packets
    - Go to search option
    - Search ARP packet in search bar.
    - Save the packets.
- Creating a filter to displaying DNS packets and provides the flow graph.
- Creating a filter of ICMP and DHCP packets and inspect and show the packets.

- Create filter for ICMP and DHCP and packet inspect and show me packets.





icmp





CamScanner

TCP





classmate
Date
Page

Arp





classmate

Date
Page

Dns





CamScanner

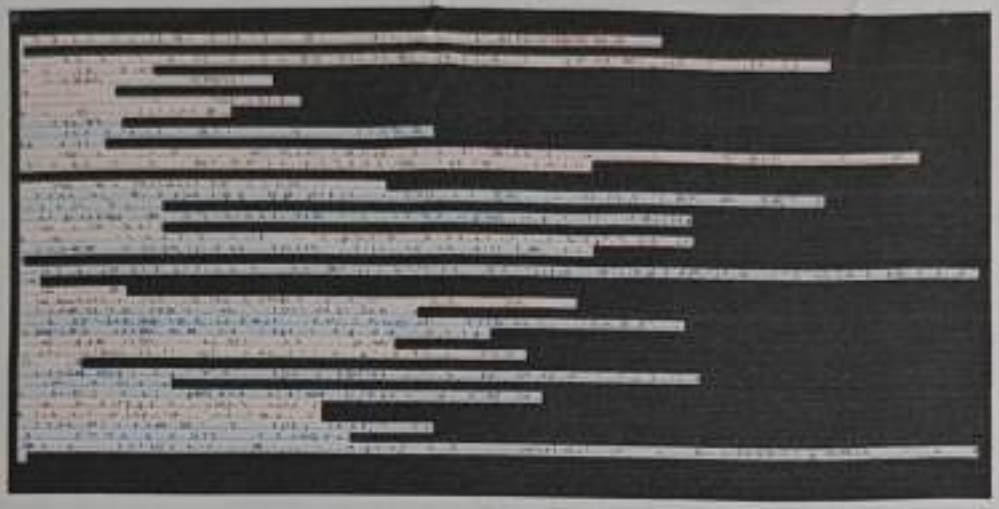
Dhcp



Flow graph



TCP Stream



Result

The experiment has been completed and executed successfully

Student Observation

1. What is promiscuous mode?

Promiscuous mode is a network-interface mode in which the network card accepts and passes packets to the packet-capture software even if the packets are not addressed to that particular computer.

2. Does ARP packet have a transport layer header? Explain

No, because ARP operates between the data link and network layers and is carried directly inside an ethernet frame.

3. Which transport layer protocol is used by DNS?

DNS primarily uses UDP, usually on port 53.

4. What is the port number used by HTTP protocol?

The standard port number for HTTP is 80.

5. What is a broadcast IP address?

A broadcast IP address is an IP address used to send a packet to all devices on a particular network/subnet.

Practical-6

Aim:
Write a program to implement error detection and correction using HAMMING code concept. Make a test run to input data stream and verify error correction feature.

~~Creater sender~~

1. Sender program

```
import math
def text_to_binary(text):
    return ''.join(format(ord(ch), '08b') for ch in text)

def hamming_encode(data):
    m = len(data)
    r = 0
    while (2**r) < (m+r+1):
        r += 1
    total_bits = m+r
    code = ['0'] * (total_bits+1)
    j = 0
    for i in range(1, total_bits+1):
        if (i & (i-1)) != 0:
            code[i] = data[j]
            j += 1
    for i in range(r):
        parity_pos = 2**i
        parity = 0
```

```
    for j in range(1, total_bits+1):
        if j & parity_pos:
            parity ^= int(code[j])
    code[parity_pos] = str(parity)
return ''.join(code[1:])

text = input("Enter text:")
binary_data = text_to_binary(text)
print("\nOriginal Text:", text)
print("Binary Data:", binary_data)

encoded_data = hamming_encode(binary_data)
print("Hamming Coded Data:", encoded_data)

with open("channel", "w") as file:
    file.write(encoded_data)
print("\nHamming coded data saved to 'channel' file.")
```

2. Receiver program:

```
def hamming_decode(code):
    n = len(code)
    bits = ['0'] + list(code)
    r = 0
    while (2**r) < n:
        r += 1
    error_position = 0
    for i in range(r):
        parity_pos = 2**i
        parity = 0
        for j in range(1, n+1):
            if j & parity_pos:
                parity ^= int(bits[j])
        if parity != 0:
            error_position += parity_pos
    if error_position != 0:
        if error_position < n:
            print("\n Error detected at
                position:", error_position)
            if bits[error_position] == '0':
                bits[error_position] = '1'
            else
                bits[error_position] = '0'
            print("Error corrected")
    else:
        print("\n Invalid error position.")
else:
    print("No error detected")
```

```
data = ""
for i in range (1, n+1):
    if (i & (i-1)) != 0:
        data += bits[i]
text = ""
for i in range (0, len(data), 8):
    byte = data[i:i+8]
    if len (byte) == 8:
        text += chr(int (byte, 2))
return text
with open ("channel", "r") as file:
    received_data = file.read().strip()
print ("Received Hamming Data:")
print (received_data)

output = hamming_decode (received_data)
print ("\n Decoded Output:")
print (output)
```

Input:
HELLO

Output:
Received Hamming Data:
[Hamming coded binary]

No error detected

Decoded output:
HELLO
Result:
The experiment has been executed and completed succesfully.

Practical 7

AIM:
Write a program to implement flow control at data link layer using sliding window protocol. Simulate the flow of frames from one node to another.

1. Sender's program

```
import time
import os
SENDER_FILE = "Sender_Buffer"
RECEIVER_FILE = "Receiver_Buffer"

window_size = int(input("Enter Window Size : "))
message = input("Enter Text Message : ")
frames = []
for i, ch in enumerate(message):
    frames.append({
        "no" : i,
        "data" : ch
    })
print("\n Total frames : ", len(frames))
print("Message :", message)

base = 0
next_frame = 0
total_frames = len(frames)
while base < total_frames:
    window_end = min(base + window_size, total_frames)
    print("Sending Frames")
    with open(SENDER_FILE, "w") as file :
```

```
for
    for i in range (base, window_end):
        frame = frames[i]
        print (
            "Frame No : ", frame ["no"],
            "DATA :", frame ["data"]
        )
        file.write (f"FRAME_NO = {frame ['no']},
                    DATA = {frame ['data']}
        )
print ("Frames saved to Sender_Buffer.")
print ("Waiting for receiver ACK ..")
time.sleep (3)
if not os.path.exists (RECEIVER_FILE):
    print ("Receiver_Buffer not found!")
    continue
with open (RECEIVER_FILE, "r") as file:
    response = file.read().strip()
print ("Receiver Response:", response)
if response.startswith ("ACK"):
    ack_no = int (response.split ("=")[1])
    print ("ACK received for Frame :", ack_no)
    if ack_no >= base:
        base = ack_no + 1
        print (" Sliding window moved.",
               "Next Frame:", base)
    else:
        print ("Old ACK received.")
```

```
    elif response.startswith("NACK"):
        nack_no = int(response.split("=")[1])
        print("\nNACK received for Frame:", nack_no)
        base = nack_no
        print("Resending frames from Frame:",
              base
        )
    else:
        print("Invalid Receiver response.")
    open(RECEIVER_FILE, "w").close()

print("All frames successfully sent")
```

2. Receiver program

```
import os
import time

SENDER_FILE = "sender_buffer"
RECEIVER_FILE = "receiver_buffer"
expected_frame = 0
print("Receiver started")
while True:
    if not os.path.exists(SENDER_FILE):
        time.sleep(1)
        continue
    with open(SENDER_FILE, "r") as file:
        data = file.readlines()
    if not data:
        time.sleep(1)
```

```
        continue

print("Frames Received")

error_found = False
for line in data:
    line = line.strip()
    if not line:
        continue
    parts = line.split(";")
    frame_no = int(parts[0].split("=")[1])
    frame_data = parts[1].split("=")[1]
    print(
        "Frame No:", frame_no,
        "DATA:", frame_data
    )
    if frame_no == expected_frame:
        print(
            "Frame", frame_no,
            "received correctly."
        )
        expected_frame += 1
    else:
        print(
            "ERROR: Expected Frame",
            expected_frame,
            "but received Frame",
            frame_no
        )
        with open(RECEIVER_FILE, "w") as file:
            file.write(f"NACK={expected_frame}")
```

```
            print (
                "NACK sent for frame:",
                expected_frame
            )
            error_found = True
            break

    if not error_found:
        ack_no = expected_frame - 1
        with open (RECEIVER_FILE, "w") as file :
            file.write (f"ACK = {ack_no}")
        print (
            "ACK sent for FRAME :",
            ack_no
        )
    time.sleep(2)
```

Input:

Enter Window Size : 3
Enter Text Message : HELLO

Output:

Total Frames : 5
Message : HELLO
Sending Frames
Frame No: 0 DATA : H
Frame No: 1 DATA : E
Frame No: 2 DATA : L

Frames saved to sender_Buffer
Waiting for Receiver ACK...

Receiver Response : ACK = 2
ACK received for FRAME : 2
Sliding Window moved Next
Frame : 3
Sending Frames
Frame No: 3 Data : L
Frame No: 4 Data : O
Frames saved to Sender_Buffer.
Waiting for Receiver ACK...
Receiver Response : ACK = 4
ACK received for Frame : 4
Sliding Window moved : Next
Frame : 5

All frames Successfully sent

Result :
The experiment has been executed and completed successfully.

PRACTICAL 8:

AIM:
To discover live hosts using Nmap scans (ARP, ICMP, TCP | UDP) on the TryHackMe platform.

Introduction
Nmap (Network Mapper) is a free and open source network scanning tool used to discover active hosts and identify services running on networked systems.

Nmap provides different techniques for discovering live hosts:

1. ARP Scan - Uses ARP requests to discover hosts on the local network.

2. ICMP scan: Uses ICMP requests and replies to identify live hosts.

3. TCP ping scan - Sends TCP packets to specified ports and checks for responses.

4. UDP ping scan - Sends UDP packets and checks for responses such as ICMP port unreachable messages.

1. NMAP HOST DISCOVERY USING ARP:

ARP stands for address resolution protocol. It is used to obtain the MAC address associated with an IP address.
When Nmap performs an ARP scan on the same subnet, it sends ARP requests to the target hosts. A host that responds to the ARP request is considered to be active.

Command:
nmap -PR -sn TARGETS

2. NMAP HOST DISCOVERY USING ICMP:

ICMP stands for Internet Control Message Protocol.
An ICMP echo request is sent to the target host. If the host is active and allows the request, it responds with an ICMP echo reply.

Command:

nmap -PE -sn TARGETS

3. TCP SYN PING:

TCP SYN PING sends a TCP packet with the SYN flag set to a specified port.
If the port is open, the target normally responds with SYN/ACK. If the port is closed, it responds with RST. In either

case, a response indicates that the host is reachable.

Command

nmap -PS80 TARGETS

4. TCP ACK PING

TCP ACK Ping sends a TCP packet with the ACK flag set.
The target may respond with a TCP RST packet, indicating that the host is active.

Command:
nmap -PA80 TARGETS

5. UDP PING

UDP Ping sends UDP packets to specified ports.
If the UDP port is closed, the target may respond with an ICMP Port Unreachable message. This response indicates that the target host is active.

6. NMAP PING SCAN

Nmap can discover live hosts without performing a port scan using:

nmap -sn TARGETS

This is useful for identifying online systems before performing further network analysis.

7. REVERSE DNS LOOKUP.

Nmap normally performs reverse-DNS lookups for online hosts to obtain hostnames.

To disable DNS resolution:

nmap -n TARGETS

To force reverse-DNS lookup:

nmap -R TARGETS

Result

The experiment has been completed and executed successfully.

PRACTICAL 9

Aim:
To implement classless IP subnetting in CISCO Packet tracer and verify connectivity between different networks using routers, switches, and PCs.

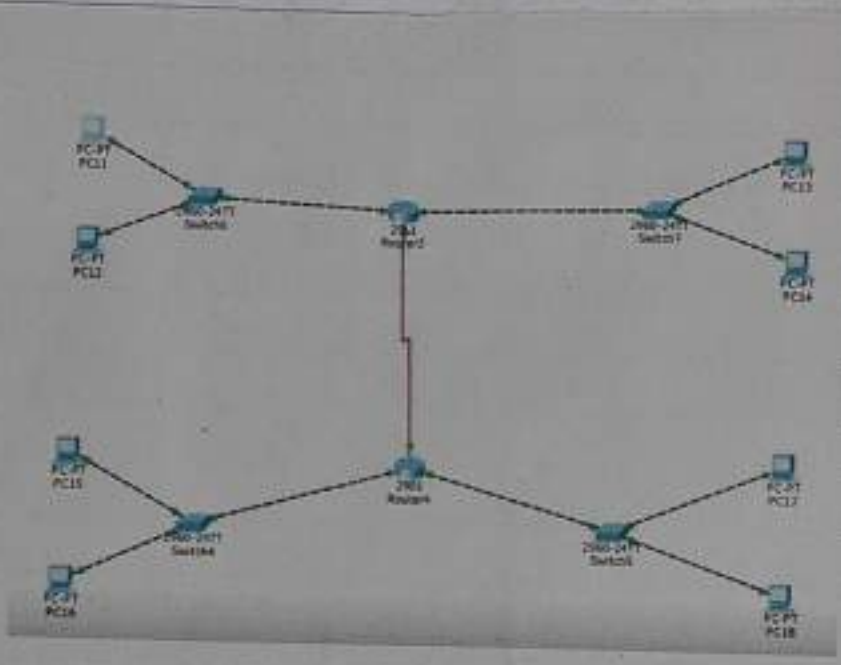
Theory

Subnetting is the process of dividing a large IP network into smaller logical networks called subnets. It helps utilize IP addresses efficiently, reduces broadcast traffic, and improves network management.
Classless inter domain routing (CIDR) is a method of IP addressing that allows variable-length subnet masks instead of using only the default masks associated with class A, class B, and class C networks.
In this experiment, 4 LANs are created using Cisco Packet Tracer. Each LAN consists of a switch and 2 PCs connected to a hub. Two routers are used to connect the LANs and enable communication b/w different networks.

Procedure

1. Open Cisco Packet Tracer
2. Place 2 routers, 4 switches and 8 PCs in the workspace.
3. Connect the PCs to the switches using copper straight-through cables.
4. Connect each switch to its respective router.
5. Connect router0 and router1 using appropriate link.
6. Configure the router interfaces.
7. Assign IP addresses, subnet masks and default gateways.
9. Configure static routes on both routers to enable communication.

9. Use the ping command to test connectivity between PCs in the same LAN and across different LANs.
10. Save the completed topology.



Result:
The experiment has been completed and executed successfully.

PRACTICAL 10.

Aim:
To design and configure a simple internetwork using a router in Cisco Packet Tracer and verify connectivity b/w two PCs using a PDU.

Theory
Internetworking is the process of connecting two or more computer networks using networking devices such as routers. It enables devices belonging to different networks to communicate with each other.

A router is a network-layer device that forwards data packets b/w different networks based on their IP addresses and routing tables.

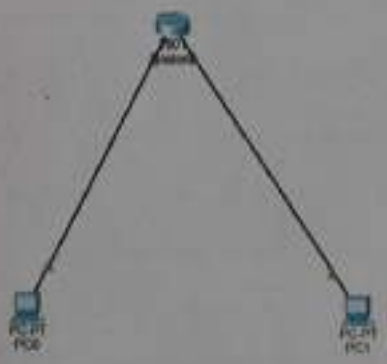
Components Used
1. Cisco 2901 Router
2. PC0 and PC1
3. Copper-Straight Through Cable
4. IP Address
5. Subnet Mask
6. Default Gateway.
7. Simple PDU

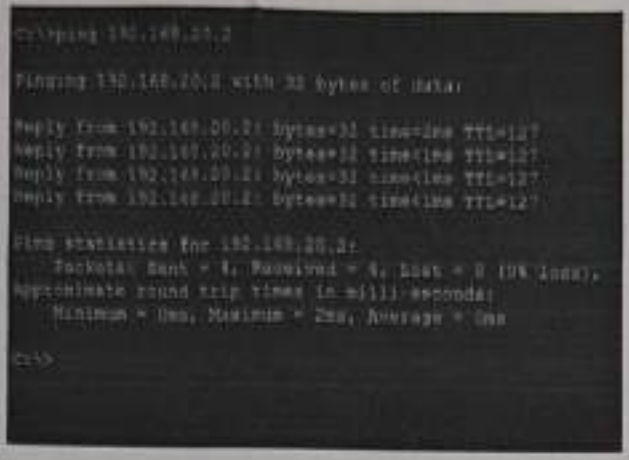
IP Address Configuration

Device	IP Address	Subnet Mask	Default Gateway
Router0-Gig0/0	192.168.10.1	255.255.255.0	—
Router1-Gig0/1	192.168.20.1	255.255.255.0	—
PC0	192.168.10.2	255.255.255.0	192.168.10.1
PC1	192.168.20.2	255.255.255.0	192.168.20.1

Working Principle:

The 2 PCs belong to different networks 192.168.10.0/24 and 192.168.20.0/24 The router connects these networks through its Gigabit Ethernet interfaces.
When PC0 sends data to PC1, it forwards the packet to its default gateway. The router examines the destination IP address and forwards the packet through the appropriate interface to PC1.
Connectivity is verified using the Internet Control message protocol (ICMP) through the simple PDU feature in packet tracer. A successful PDU indicates that communication between the 2 PCs is working correctly.





Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC0	PC1	ICMP		0.000	N	0	(edit)	(delete)

Result

The experiment has been completed and executed successfully.

PRACTICAL 11.

Aim:
To simulate the routing information Protocol (RIP) using Cisco Packet Tracer and study dynamic routing between interconnected networks.

Theory
RIP is a dynamic routing protocol used by routers to exchange routing information and determine the best path for forwarding data packets from a source network to a destination network.

Types of RIP
- RIPv1
- RIPv2

Working Principle
RIP works by exchanging routing information between neighboring routers. Each router maintains a routing table containing destination networks.
When RIP is enabled, routers advertise their directly connected networks to neighbouring routers. They update their routing tables based on the received information and select the route with the lowest hop count.

Commands Used

- enable
- configure terminal
- interface
- ip address

- no shutdown
- exit
- ping
- tracert

Advantage of RIP

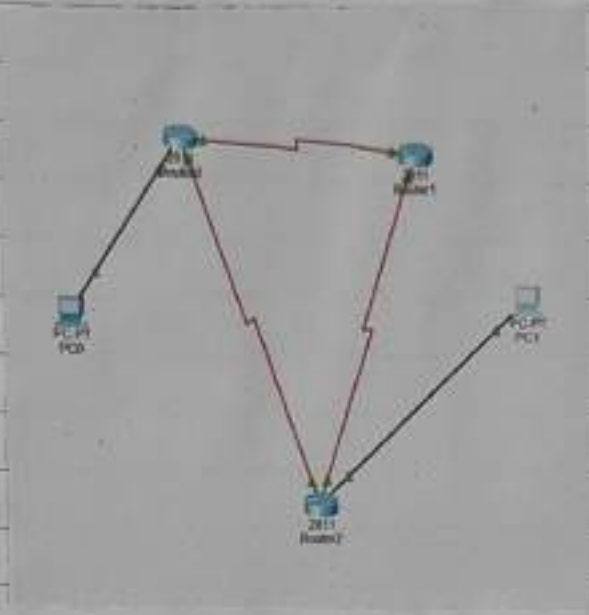
1. Simple to configure and maintain
2. Automatically exchanges routing information
3. Can adapt to network topology changes.

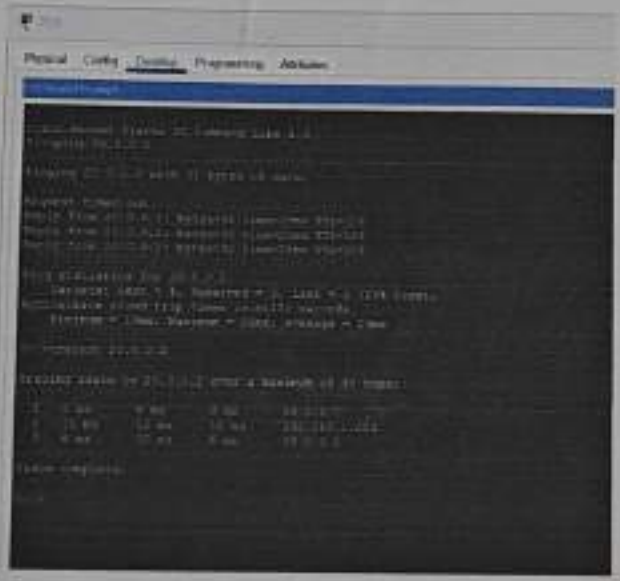
Limitations of RIP

1. Supports a maximum of 15 hops
2. Hop counts does not consider bandwidth or delay.
3. It is less suitable for large and complex networks.

Procedure

1. Open Cisco Packet Tracer and place 3 routers and 2 PCs in the workspace.
2. Connect the devices using suitable ethernet and serial cables.
3. Assign IP addresses and subnet masks to all PCs and router interfaces.
4. Enable the interfaces using the no shutdown command.
5. Configure RIP version 2 on all 3 routers.
6. Advertise the directly connected networks using the network command
7. Verify the routing tables using show ip route.
8. Test communication b/w PC0 and PC1 using ping command
9. Use tracert to identify the path followed by the packets.
10. Disconnect the direct link b/w router0 and router2 and observe whether communication resumes through Router1


PC-PT
PC0
PC-PT
PC1


Physical
Config
Desktop
Programming
Attributes



Result

The experiment has been executed and completed successfully.

Practical 12a

Aim:
To implement an echo client - server application using TCP/UDP sockets.

Algorithm
1. Start the program
2. Create a socket for the server using TCP or UDP.
3. Assign an IP address and port number to the server.
4. For TCP, bind the socket, listen for client connections, and accept the connection. For UDP, bind the socket and wait for a message.
5. Create a client and connect to the server in TCP or send a message to the server in UDP.
6. The client sends a message to the server.
7. The server receives the message and sends the same message back to the client.
8. The client receives and displays the echoed message.
9. Close the sockets and terminate the program.

Input
A message or string entered by the client

Example: Hello Server

Output:
The server echoes the same message back to the client.

Example: Echo from Server : Hello Server.

Result:

The experiment was implemented successfully

CamScanner

Practical 12b.

Aim:
To implement a chat client-server application using TCP/UDP sockets.

Algorithm:

Server:
1. Start the program and create a socket.
2. Bind the socket to the server IP address and port number.
3. For TCP, listen for incoming client connections and accept
4. For UDP, wait to receive messages from the client
5. Receive messages from the client and display them.
6. Send a reply message to the client.
7. Repeat the communication until the client disconnects or the chat ends
8. Close the socket and terminate the program.

Client
1. Start the program and create a socket.
2. Specify the server IP address and port number.
3. For TCP, connect to the server.
4. For UDP, send messages directly to the server address
5. Enter a message and send it to the server.
6. Receive and display the server's reply.
7. Repeat the process until the chat ends
8. Close the socket and terminate the program.

Input

- Server IP address and port number
- messages entered by the client and server.
- Communication protocol : TCP or UDP.

Output:

- Successful connection or communication b/w the client and server.
- Messages sent by the client are received and displayed by the server.
- Replies from the server are received and displayed by the client.
- Chat terminates when the communication is stopped.

Result:

The experiment is executed and completed successfully.

PRACTICAL 13 :

Aim:
To implement a ping program to check the connectivity b/w a source computer and a destination host

Algorithm:
1. Start the program.
2. Import the required networking libraries.
3. Get the destination IP address or hostname from the user.
4. Send an ICMP echo request packet to the destination host.
5. Wait for the ICMP echo reply from the destination.
6. Calculate and display the response time if a reply is received.
7. Display an error message if the destination is unreachable or no reply is received
8. Stop the program.

Input:
Destination IP address or hostname

Output:
- Displays the destination host's IP address
- Displays the reply status and response time.
- Displays an error message if the destination is unreachable.

Result:
The experiment has been executed and completed successfully.

PRACTICAL 14

AIM:

To write a program using raw sockets to implement packet sniffing.

Algorithm:

- Start the program.
- Create a raw socket to capture network packets
- Bind the socket to the required network interface, if necessary.
- Receive packets from the network using the raw socket.
- Extract and display packet details such as source IP address, destination IP address, and protocol type.
- Continue capturing packets until the user terminates the program.
- Close the socket and terminate the program.

Input:

Network packets received through the network interface.

Output:

Displays the captured packet details, including source IP address, destination IP address, and protocol type.

Result:

The experiment was executed and completed successfully.

12. End - End communication at transport layer.

a) Implement echo client server using TCP/UDP & sockets.
b) Implement a chat program using socket programming.

TCP

```
import socket
import threading
def start_server():
    server = socket.socket(socket, AF_INET, socket.SOCK_STREAM)
    server.bind('127.0.0.1', 5000))
    server.listen()
    print("[SERVER] Listening on 127.0.0.1:5000")
    conn, addr = server.accept()
    print(f"[server] connected by {addr}")
    while True:
        data = conn.recv(1024).decode()
        if not data or data.lower() == 'quit':
            print("[server] connection closed by client.")
            break
        print(f" [SERVER] Received: {data}")
        conn.send(f" Server received: {data}".encode())
    conn.close()
    server.close()

def start_client():
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect('127.0.0.1', 5000))
    print("[client] connected to server")
```

```
while True:
    msg = input("Enter message (or 'quit' to exit): ")
    client.send(msg.encode())
    if msg.lower() == 'quit':
        break
    response = client.recv(1024).decode()
client.close()

if __name__ == "__main__":
    server_thread = threading.Thread(target = start_server,
        daemon = True)
    server_thread.start()
    start_client()
```

Output:

[SERVER] listening on 127.0.0.1 5000 [CLIENT] connected to server

[SERVER] connected by ('127.0.0.1', 53458) Enter message (or 'quit' to exit):

hello

[SERVER] Received: hello

UDP:

```
import socket
import threading

def udp_server():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.bind(('127.0.0.1', 12000))
    print("[SERVER] Running on 127.0.0.1 12000")
    while True:
        data, addr = s.recvfrom(1024)
        msg = data.decode()
        if msg.lower() == 'quit':
            print(f"[SERVER] Client {addr} ended the
        connection.")
        break
        print(f"[SERVER] Received from {addr}: {msg}")
        s.sendto(data, addr)
    s.close()

def udp_client():
    c = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server = ('127.0.0.1', 12000)
    while True:
        msg = input("Enter message")
        c.sendto(msg.encode(), server)
        if msg.lower() == 'quit':
            print("[CLIENT] Closing connection...")
    c.close()
```

```
if __name__ == "__main__":
    threading.Thread(target = udp_server,
                     daemon = True).start()udp_client()
```

Output:

```
Enter message : [SERVER] running on 127.0.0.1:12000
[SERVER] Received from ('127.0.0.1', 49670): hi
Echo from Server : hi
[Client] Stopped. [SERVER] Stopped
```

5. Implement a ping program.

```
import os
import platform
import subprocess

def ping(host = "google.com", count = 4):
    param = '-n' if platform.system().lower() == "windows"
                                        else "-c"
    command = ["ping", param, str(count), host]
    try:
        output = subprocess.check_output(command,
                                universal_newlines = True)
        print(output)
    except Exception as e:
        print("Ping failed")

if __name__ == "__main__":
    ping("8.8.8.8")
```

Output:

Pinging 8.8.8.8 with 32 bytes of data

Reply from 8.8.8.8 : bytes = 32 time = 6ms TTL = 119.

Reply from 8.8.8.8 : bytes = 32 time = 9ms TTL = 119.

Ping statistics from 8.8.8.8.

Minimum = 5ms , Maximum = 9ms , Average = 7ms.

14. Raw sockets to implement packet sniffing.

```
from scapy.all import sniff
from scapy.layers.inet import IP, TCP, UDP, ICMP.

def packet_callback(packet):
    if IP in packet:
        ip_layer = packet[IP]
        protocol = ip_layer.proto
        src_ip = ip_layer.src.

        protocol_name = ""
        if protocol == 1:
            protocol_name = "ICMP"
        elif protocol == 6:
            protocol_name = "TCP"
        elif protocol == 17:
            protocol_name = "UDP"
        else:
            protocol_name = "Unknown".

        print(f" Protocol : {protocol_name}")
        print(f" Source IP: {src_ip}").
        print("-"*50).
sniff(iface='WiFi', prn=packet_callback, filter="ip", store=0)
```

Output:

Protocol: TCP
Source IP : 57.155.141.117
Destination IP : 192.168.1.6

Protocol : TCP
Source IP : 192.168.1.6
Destination IP : 57.155.141.117

Protocol : TCP
Source IP : 192.168.1.6
Destination IP : 57.155.141.117.

Completed